Figure 6.12 Path compression in tries, by merging together edges adjacent to pass-through nodes; notice that edges in radix tries are labeled with strings, not just characters.

The original trie has 9 nodes and 8 edges, and with the assumptions made in section 6.2.1, with an 4-bit overhead per node, this means 9 * 4 + 8 * 9 = 108 bytes.

The compressed trie on the right has 6 nodes and 5 edges, but in this case each edge carries a string, not just a character; however, we can simplify the operation by accounting for edge references and string labels separately: this way, we would still count 9 bytes per edge (because we would include the string terminator byte in the edge cost), but we could add the sum of strings lengths as a third term in the sum; the total number of bytes needed is given by 6 * 4 + 5 * 9 + 8 * 1 = 77 bytes.

In other words, for this simple trie, the compressed version requires 30% less memory.

6.3.1 Nodes and Edges

All the operations that we have described for tries can similarly be implemented for radix trees: just instead of edges labeled by chars, we need to store, and follow, edges labeled by strings.

While at a high level the logic of the methods is almost the same as for trie, to check which branch we should traverse we can't just check the next character in the key, as edges